

WimPyC Documentation

Sunghyun Kang, Stefano Scopel, Gaurav Tomar

Department of Physics, Sogang University, South Korea

Center for Quantum Spacetime, Sogang University, South Korea

School of Basic Sciences and Humanities, APJ Abdul Kalam Technological University, India

To get more details and follow updates, please visit our homepage:
wimpydd.hepforge.org

Classes-

- `isotope(object)`

Initializes a single nuclear isotope extracting it from an object belonging to the "element" class. Isotope objects are needed as input by the following routines:

`WD.load_response_functions_capture`

`WD.plot_response_functions_capture`

If the "element" object is not passed as a parameter it is searched among the set of elements that is pre-defined in WimPyDD:

Al, Ar, C, Ca, F, Fe, Ge, H, He, I, Mg, N, Na, Ne, Ni, O, S, Si, W, Xe that can also be listed typing `list_elements()`. Requires as input a string with the symbol of the isotope, that must match one of the entries of the "isotopes" attribute of the element object.

Example:

```
Xe128=WD.isotope('128Xe')
```

successfully initializes the Xe128 isotope if the predefined element `WD.Xe` exists, and if its `WD.Xe.isotopes` attribute contains the string '128Xe':

```
>>>'128Xe' in WD.Xe.isotopes
```

```
True
```

If the isotope belongs to an element that is not pre-defined one needs first to initialize a new element object (see `help(WD.element)` and Appendix D of I, Jeong et al., Computer Physics Communications 276(2022)108342 for instructions on how to do it) and then pass it as an argument to the isotope class.

For instance:

```
>>> Cl35=WD.isotope('35Cl')
```

looking for 35Cl in list of pre-defined targets:

35Cl not found, check isotope spelling. Returned empty isotope object with success attribute equal to False.

The element object where to find the isotope can be passed as an argument.

```
>>>cl=WD.element('Cl')
```

```
'35Cl' in cl.isotopes
True
>>>cl.name
chlorine
>>>Cl35=WD.isotope('35Cl',element=cl)
35Cl found in chlorine
```

Parameters:

symbol(str) - A string that identifies an isotope. By convention should start with an atomic mass number followed by the capitalized symbol of the element. Must match one of the entries of the "isotopes" attribute of the "element" object input (see below)

Example: `H1=WD.isotope('1H')` requires that the predefined element `WD.H` contains the string '1H' in its isotopes attribute:

```
WD.H.isotopes → array(['1H', '2H'])
```

element(class element) - The object belonging to the element class where the isotope is contained. If None, the code will search among the pre-defined elements in `WimPyDD` (type `WD.list_elements()` to list them). Default=None

verbose(bool) - If True prints out details of the process that searches for the isotope. Default=True

Attributes:

a(integer) - atomic mass numbers of the isotope

element(element object) - the element object from where the isotope has been extracted

itar(integer) - internal code to access the default Nuclear functions coefficients (see element object)

mass(float) - mass of the isotope in GeV

spin(float) - spin of the isotope

symbol(str) - symbol of the isotope

z(int) - atomic number of the isotope

response_functions_capture(dict) - a dictionary containing the tabulated response functions loaded by the `load_response_functions_capture` routine (see help (`WD.load_response_functions_capture` routine) for details). contains the corresponding tabulated `sigma_tilde` used by `WD.wimp_capture_rate` for the calculation of WIMP capture rate signals in celestial bodies. For a definition of the integrated response functions `sigma_tilde` see WimPyC paper(arXiv:2510.21185)

- **celestial_body(object)**

Instantiates a celestial body object containing the astrophysical information required to calculate the WIMP capture rate: radius and mass of the system, target nuclei, densities. All the astrophysical information must be collected in a subfolder of the `WimPyDD/Celestial.bodies` directory matching the string name passed as the first argument. Such folder must contain the following user-provided files:

```
densities.tab
```

radius.tab

mass.tab

Example:

```
sun=WD.celestial_body('Sun')
```

Subfolder:

WimPyDD/WimPyC/Celestial.bodies/Sun

The mass and radius must be in solar units, so in the case of the Sun the two files mass.tab and radius.tab will contain 1. The structure of the densities.tab file is an optional first line starting with “#” containing help information (returned by help(sun)) followed by headers and by a series of columns containing the radial dependence of nuclear densities. The headers of the file must include 'r_vec', 'rho_tot' and at least one recognizable isotope name (see help(WD.isotope)). if 'v_esc' is missing it will be calculated using the radial density. In the r_vec, rho_tot and rho_i attributes the radius is normalized to 1 and the densities are normalized to a unit mass (i.e. $\int r^2 dr \rho_{tot} = 1$) using the information contained in radius.tab and mass.tab. So in densities.tab the units of the radius and of all densities are arbitrary (nuclear densities must only have the same units as the total density 'rho_tot'). The only dimensional quantity in the file is v_esc, which, if present, must be in km/s.

Example:

```
# Solar Model AGSS09ph, A. Serenelli et al, Astrophys. J. Lett. 705 (2009) L123–L127, [0909.2668].
r_vec rho_tot 1H 4He ..... v_esc
0.0015 150.5 74.63614626654312 130.65634553820425 ..... 1339.5764710930496
0.002 150.5 67.19046650038389 117.57028777936976 ..... 1339.6029805466728
.....
0.9815 0.001162 0.0009100800407837143 0.00028441817946676305 ..... 617.4823945410445
```

Parameters:

name(str) - name of celestial body. Corresponds to the name of the subfolder in the Celestial_bodies folder containing the files with the astrophysical input.

verbose(bool) - If True print out the details of the loading procedure from the input directory. Default: False

Attributes:

name(str) - celestial body name.

mass(float) - celestial body mass in solar units.

r_vec(array) - radius interval array for densities of isotopes (normalized to 1).

radius(float) - celestial body radius in solar units.

rho_i(dict) - dictionary of densities for each isotope normalized to the isotope total mass fraction.

Example: sun.rho_i['4He'] contains the radial density profile of Helium 4 for each value of the radius contained in sun.r_vec $\int sun.r_vec, 4 * np.pi * sun.rho_tot * sun.r_vec ** 2 \rightarrow 0.26$

rho_tot(array) - density profile of the total radial density of the celestial_body for each value of the radius contained in sun.r_vec normalized to a unit total mass:

$\int sun.r_vec, 4 * np.pi * sun.rho_tot * sun.r_vec ** 2 \rightarrow 1$

target_names(array) - an array containing the names of the target isotopes inside the celestial body (loaded from the headers of the densities.tab file).

`targets(array)` - an array of objects belonging to the isotope class corresponding to the targets inside celestial body.
`v_esc(array)` - an array of escape velocities for each value of the radius contained in `sun.r_vec`. In km/sec.

Functions-

All the functions provided with the WimPyC are briefly described as following,

- `plot_response_functions_capture(input_obj,model,j_chi=0.5,coeff_squared_list=None,style='',linewidth=1,tuple=None,n_vel_list=None,tau_list=None,tau_prime_list=None,rescaling_factor=1.,scatter_plot=False,scatter_plot_point_size=20,scatter_plot_color=None,er_min_cut=1e-6,er_max_cut=None,**args)`

`input_obj` can be a nuclear isotope or a celestial body.

Plots the response functions contained in `isotope.response_functions[hamiltonian,j_chi]`

Parameters:

`isotope or celestial body` - object belonging either to the isotope class or to the `celestial_body` class

`model` - object belonging to `eft_hamiltonian` class

`j_chi` - WIMP spin. Default: 0.5

`coeff_squared_list` - A list of the Wilson coefficients combinations for which the response functions are plotted (among those in `hamiltonian.coeff_squared_list`. If None, all are plotted. Default: None

Example: `[('0omega', 0, 0), ('0omega', 0, 0)]`

`[(4,4), (6,6)]`

Example: if `o4.o6.coeff_squared_list -> [(4, 4), (4, 6), (6, 4), (6, 6)]`,

`plot_response_functions_capture(Sun.targets[0],o4_o6,j_chi=0.5)`

total of 64 response functions

16 non-vanishing response functions plotted

`plot_response_functions(Sun.targets[0],o4_o6,n_vel_list=[0],tau_list=[0],tau_prime_list=[1],coeff_squared_list=[[6,6]])`

plots a single response function for `c_6,c_6`, `n_vel=0` (velocity-independent term in squared amplitude), `tau=0`, `tau_prime=1`

Output:

Figure showing response functions for defined experiment `exp` as a function of recoil energy

-
- `load_response_functions_capture(input_obj, hamiltonian, j_chi=0.5, reset=False, verbose=True, update_time_stamp=False, n_sampling=1000, force_recalculation=False, u_esc_max=2000, er_min_cut=1e-6, er_max_cut=None, er_sampling=None, increase_sampling=False, **args)`

For the effective Hamiltonian `hamiltonian` and the WIMP spin `j_chi` loads the differential response functions `d_sigma_tilde_der` and loads or calculates the tabulated response functions `sigma_tilde` (for their definition see WimPyC paper(arXiv:2510.21185)) used by `wimp_dd_capture` to calculate the WIMP capture rate in a celestial body.

If `input_obj` belongs to the `isotope` class it updates the dictionaries `input_obj.diff_response_functions` and `input_obj.response_functions` for the single isotope.

If `input_obj` belongs to the `celestial_body` class it updates the dictionaries of all the isotopes contained in `input_obj.targets`

Parameters:

`input_obj` - object belonging to `celestial_body` class or `isotope` class.

`hamiltonian` - object belonging to `eft_hamiltonian` class.

`j_chi(float)` - WIMP spin. Default: 0.5

`reset(bool)` - Empties the `isotope.response_functions` dictionary before adding the output. Default: False

`verbose(bool)` - If True prints out a list of the response function tables that are loaded or written. Default: True

`update_time_stamp(bool)` - If true updates the time stamp of all response functions tables. Default: True

`n_sampling(int)` - The number of points of response functions sampling. Default: True

`force_recalculation(bool)` - If True delete saved files of response function before calculation. Default: False

`u_esc_max(float)` - maximum escape velocity inside the celestial body.

`er_min_cut(float)` - minimum recoil energy inside the celestial body.

`er_max_cut(float)` - maximum recoil energy inside the celestial body.

`er_sampling` - used only if `increase_sampling=True`. A list of energy values in keV used to increase the sampling of the response functions. Default: None

`increase_sampling` - if True the sampling of the response functions is increased using the energy values contained in the `er_sampling` parameter. Ignored if `er_sampling` parameter=None.

Output:

if `input_obj = isotope_obj`:

- add the entry `{(hamiltonian_obj.name, j_chi, tau, tau_prime): func_array}` to the dictionary `isotope_obj.diff_response_functions_capture`

- adds the entry `{(hamiltonian_obj.name, j_chi): r}` to the dictionary `isotope_obj.response_functions_capture`

if `input_obj= celestial_body_obj` repeats the same procedure for every target in `celestial_body_obj`.

func_array: array of differential response functions with shape (2,2,4) corresponding to tau, tau_prime and n_vel with tau,tau_prime=0,1 the nuclear isospin and n_vel=0,1,2,3 corresponds to the amplitude decomposition 0,1,1E, 1E⁻¹ (see help of `eft_amplitude_squared`).

Structure of tuple r:

Setting:

n_cicj=0,...,len(hamiltonian_obj.coeff_squared_list)-1 = entry of the array `hamiltonian_obj.coeff_squared_list` containing couplings pairs

n_vel=0,1,2,3 corresponding to a=0,1,1E,1E⁻¹ (amplitude decomposition, see help on the `eft_amplitude_squared` routine)

tau=0,1 (nuclear isospin)

tau_prime=0,1 (nuclear isospin)

the tuple entry:

`r[n_cicj][n_vel][tau][tau_prime][0]`

contains a sampling of recoil energy values in keV

and:

`r[n_cicj][n_vel][tau][tau_prime][1]`

contains the corresponding tabulated integrated response functions `sigma_tilde` used by `WD.wimp_capture_rate` for the calculation of WIMP capture rate signals in celestial bodies. For a definition of the `sigma_tilde` response functions see WimPyC paper(arXiv:2510.21185).

N.B. Notice that the the response functions are an attribute of the nuclear isotopes and not of the celestial bodies. In other words, two celestial bodies containing the same isotope share the same response functions.

Example:

having defined the isotope Al27 and the hamiltonian h:

```
>>> print(Al27)
```

symbol 27Al, atomic number 13, mass 25.137

Nuclear form factor:Definition from Phys.Rev.C 89 (2014) 6, 065501 (e-Print: 1308.6288[hep-ph]) used for nuclear W functions as default

```
>>> print(h)
```

Hamiltonian name:h

Hamiltonian:c_4()* O_4+c_5()* O_5+c_6()* O_6

Squared amplitude contributions:

O_4*O_4, O_4*O_6, O_5*O_4, O_5*O_5, O_6*O_4, O_6*O_6

the command line:

```
>>> WD.load_response_functions_capture(Al27, h, j_chi=0.5)
```

calculates for a WIMP of spin=0.5 (default) the response functions `sigma_tilde` for Al27 or loads them from WimPyDD/WimPyC/Response.functions/spin_1.2 and stores them in the `Al27.response_functions_capture` dictionary. The `sigma_tilde` corresponding to the c5*c4 combination of effective couplings, to the amplitude component 0 and to the isospin combination [tau,tau_prime]=[0,1] is in:

```
>>> r=Al27.response_functions_capture[h.name,0.5]
```

```
>>> n_cicj=2
```

because:

```
>>> h.coeff_squared_list[n_cicj=2]
```

```
(5, 4)
```

```

n_vel=0 (with the correspondence 0 → "0", 1 → "1", 2 → "1E", 3 → "1E^-1",
see help(WD.eft_amplitude_squared))
tau=0
tau_prime=1
energy=r[n_cicj][n_vel][tau][tau_prime][0]
sigma_tilde=r[n_cicj][n_vel][tau][tau_prime][1]

```

- `wimp_capture(celestial_body_obj,hamiltonian,vmin,delta_eta,mchi,j_chi=0.5, delta=0,rho_loc=0.3,response_functions=None,targets_list=None, coeff_squared_list=None,reset_response_functions=False,print_contributions=False, tau_range=range(2),tau_prime_range=range(2),n_vel_list=range(4),sum_over_streams=True,verbose=False,v_cut=0,choose_zero=True,increase_sampling=False, response_functions_er_sampling=None,**args)`

Calculates the WIMP capture rate in a celestial body. The routine `load_response_functions_capture` is called if `isotope.response_functions` does not contain the required response functions for `hamiltonian` and `j_chi`.

Parameters:

`celestial_body_obj` - object belonging to `celestial_body` class
`hamiltonian` - object belonging to `eft_hamiltonian` class
`vmin` - an array containing a list of WIMP asymptotic speeds in km/s.
`delta_eta` - an array containing the contribution of each stream to the halo function $\eta(v)$ in $(\text{km/sec})^{-1}$ (the routine `streamed_halo_function` can be used to calculate it)
`mchi` - WIMP mass in GeV
`j_chi` - WIMP spin. Default: 0.5
`delta` - Mass splitting for inelastic scattering in keV. Default: 0
`rho_loc` - local dark matter density in units of GeV/cm^3 . Default: 0.3
`response_functions` - if passed, overrides the set of response functions. Must be a tuple with the same format of `isotope.response_functions[hamiltonian,j_chi]`. Default: None
`targets_list` - a list of objects belonging to the `isotope` class to override the target list contained in `celestial_body_obj.targets`. If None the targets in `celestial_body_obj.targets` are used. Default: None
`coeff_squared_list` - a list of Wilson coefficients combinations used to calculate the capture rate (same as the attribute `coeff_squared_list` of the `eft_hamiltonian` class) Default: None
`reset_response_functions` - If True empties the dictionary `isotope.response_functions` and reloads the response functions for `hamiltonian` and `j_chi`. Default: False
`print_contributions` - If True, prints out the contributions to the capture rate from each combination of the Wilson coefficients. After each call to `wimp_capture` such contributions can be access via the dictionary:
`wimp_capture.dc_dvi[(target.symbol,[c2,c2])]` with `[c1,c2]` any couplings combination contained in `hamiltonian.coeff_squared_list`. Default: False
`tau_range, tau_prime_range` - overrides the values of `tau, tau_prime` in the double sum over nuclear

isospins $\sum_{\tau} \sum_{\tau'} R^{\tau \tau'} W^{\tau \tau'}$ that enters the calculation of the rate. It allows to calculate the contribution to the rate of specific isospin combinations. Default: range(2)

`n_vel_list` - a list of values 0,1,2,3 corresponding to $a=0,1,1E,1E^{-1}$ (amplitude decomposition. see help on `eft_amplitude_squared` routine). Default: range(4)

`sum_over_streams` - if False does not sum over velocity streams and returns for each target isotopes the contributions to the rate from each stream, in an array with shape `vmin.shape + r_vec.shape`. Default: True

`verbose` - Passed to `load_response_functions`, if called. Default: False

`v_cut` - `v_cut` parameters that controls the maximal aphelium that the bound orbit of a WIMP after scattering must have in order to be captured. Introduced to take into account gravitational disturbances of other celestial bodies. For details, see 1308.5897, 1204.5120, 1305.0912, 2408.09658. See also help of `er_capture` routine. Default: 0

`choose_zero` - If True set `dc_dvi` as zero when `e_max` is smaller than `e_min` Default: True

`increase_sampling` -if True the sampling of the response functions is increased using the energy values contained in the `response_functions_er_sampling` parameter. Default: False

`response_functions_er_sampling` - list of energy values in keV used to increase the sampling of the response functions if `increase_sampling` is True. If None a list of energy values is obtained using the `get_wimp_capture_response_functions_energy_sampling` routine (see dedicated help). Default: None

Output:

The value of capture rate in sec^{-1}

Example:

```
sun=celestial_body('Sun')
o1=eft_hamiltonian('o1',1:lambda:[1,1])
load_response_functions_capture(sun,o1,j_chi=0.5)
vmin,delta_eta0=streamed_halo_function() (time-independent component of halo function for
Maxwellian with default values of parameters - see help on streamed_halo_function)
wimp_capture(sun,o1,vmin,delta_eta0,mchi=100)
gives capture rate
```

-
- `er_capture(mchi,delta,vmin,v_cut=0)`

Calculates the minimum energy that the WIMP needs to lose in order to be captured.

Parameters:

`mchi` - WIMP mass in GeV

`delta` - mass splitting in keV

`vmin` - incoming WIMP speed in km/s

`v_cut` - in km/s. If different from zero counts as “captured” only dark matter particles which are kinematically constrained to orbits with maximum radius `r0`. Indicating with $v_{\text{esc}}(r \rightarrow \text{infinity})$ the

escape velocity from radius r and with $v_{\text{esc}}(r \rightarrow r_0)$ the velocity to escape from radius r to radius r_0 the velocity v_{cut} is given by the constant value $v_{\text{cut}}^2 = v_{\text{esc}}(r \rightarrow \text{infinity})^2 - v_{\text{esc}}(r \rightarrow r_0)^2$. For instance, for $r_0 = \text{Sun-Jupiter distance}$ one has $v_{\text{cut}} = 18 \text{ km/s}$. Setting $v_{\text{cut}} = 18 \text{ km/s}$ implies to count as captured only dark matter particles confined to orbits that lie within Jupiter's orbit (see Eq.(8) of Kumar et al., PRD86 (2012) 073002, (arXiv:1204.5120)). Default: 0

Output:

the value of minimum energy in keV that the WIMP needs to lose in order to be captured.

- `wimp_capture_matrix(celestial_body_obj, hamiltonian, vmin, delta_eta, mchi, delta=0, j_chi=0.5, targets_list=None, verbose=False, **args)`

Outputs the matrix M that calculates the capture rate C for a couplings array c :

$C = \text{np.dot}(c, \text{np.dot}(M, c))$

for a given celestial body, hamiltonian, and velocity distribution. A dictionary with the mapping between the array components of c and the couplings of the hamiltonian is provided by the routine `get_mapping`.

N.B. The output matrix M of `wimp_dd_matrix` is always in the isospin base. The M_{pn} in the proton-neutron base can be obtained using:

$U = \text{WD.rotation_from_isospin_to_pn}(\text{hamiltonian})$

$M_{\text{pn}} = \text{np.dot}(U, \text{np.dot}(M, U))$

Parameters:

`celestial_body_obj` - object belonging to the `celestial_body` class

`hamiltonian` - object belonging to `eft_hamiltonian` class

`vmin` - array containing a list of WIMP stream speed velocities in the lab frame in km/s.

`delta_eta` - array containing the contribution of each stream to the halo function $\eta(v)$ in $(\text{km/sec})^{-1}$ (the routine `streamed_halo_function` can be used to calculate it)

`mchi` - WIMP mass in GeV

`delta` - Mass splitting for inelastic scattering in keV. Default: 0

`j_chi` - WIMP spin. Default: 0.5

`targets_list` - a list of objects belonging to the `isotope` class to override the list of targets contained in `celestial_body_obj.targets`. If None the targets in `celestial_body_obj.targets` are used. Default: None

`verbose` - Passed to `load_response_functions`, if called. Default: False)

Output:

array M with shape $(2 \cdot \text{len}(\text{hamiltonian.couplings}), 2 \cdot \text{len}(\text{hamiltonian.couplings}))$

(the dimensionality corresponds to twice the number of couplings to include two values of the nuclear isospin).

Given a set of coupling $c = (c_1, c_2, \dots, c_n)$ the quantity $c^T M c$ is the capture rate in sec^{-1}

Example:

```
>>> c_1_3=WD.eft_hamiltonian('c_1_3',{1: lambda: [1,1],3: lambda: [1,1]})
>>> mapping=WD.get_mapping(c_1_3)
>>>mapping[3,0]
>>>2
the index of the coupling c_3^0 is 2.
>>>sun=WD.celestial_object('Sun')
>>>m=WD.wimp_capture_matrix(sun,c_1_3,n_bin,vmin,delta_eta,mchi)
```

- `wimp_capture_accurate(celestial_body_obj,hamiltonian,vmin,delta_eta,mchi, j_chi=0.5,delta=0,rho_loc=0.3,response_functions=None,targets_list=None,coeff_squared_list=None,reset_response_functions=False,tau_range=range(2),tau_prime_range=range(2),n_vel_list=range(4),sum_over_streams=True,verbose=False,flatten=False,v_cut=0,choose_zero=True,**args)`

Routine alternative to `wimp_capture` that calculates the WIMP capture rate without making use of the interpolation method. WARNING: significantly slower compared to `wimp_capture`

Parameters:

All its input arguments are in common with the `wimp_capture` routine

Output:

The capture rate in s^{-1}

- `get_wimp_capture_response_functions_energy_sampling(target, celestial_body_list, mchi_list,delta_list,vmin, n_sampling=1000,**args)`

calculates an optimized list of energy values in keV that can be used to increase the sampling of the integrated response functions `sigma_tilde`. See WimPyC paper(arXiv:2510.21185) for details of how the optimization is performed.

Parameters:

`target` - object belonging to the isotope class

`celestial_body_list` - list or array of objects belonging to the `celestial_body` class

`mchi_list` - list or array of values of the WIMP mass in GeV

`delta_list` - list or array of value of the mass splitting δ in keV

`vmin` - list or array of WIMP asymptotic speeds (at large distance from the celestial body) in km/s

`n_sampling` - (approximate) length of output array

Output:

array of energy values in keV

- `wimp_capture_geom(celestial_obj, mchi, vmin, delta_eta, rho_loc=0.3)`

calculates the geometrical capture rate

Parameters:

`celestial_obj` - object belonging to `celestial_body` class

`mchi` - WIMP mass in GeV

`vmin` - an array containing a list of WIMP asymptotic speeds in km/s.

`delta_eta` - an array containing the contribution of each stream to the halo function $\eta(v)$ in $(\text{km/sec})^{-1}$ (the routine `streamed_halo_function` can be used to calculate it)

`rho_loc` - local dark matter density in units of GeV/cm^3 . Default: 0.3

Output:

WIMP geometrical capture in sec^{-1}

- `wimp_capture_annihilation(celestial_body_obj, mchi, capture_rate, sigma_v=3e-26, t_age=4.603e9, effective_volume=None)`

calculates the WIMP annihilation rate following K. Griest and D. Seckel, Cosmic Asymmetry, Neutrinos and the Sun, Nucl. Phys. B 283 (1987) 681–705.

Parameters:

`celestial_obj` - object belonging to `celestial_body` class

`capture_rate` - wimp capture rate in sec^{-1}

`mchi` - WIMP mass in GeV

`t_age` - age of the celestial body in years. Default value: 4.603e9 (age of the Sun)

`sigma_v` - WIMP annihilation cross section times velocity ($\lim v \rightarrow 0$). Default: $3\text{e-}26 \text{ cm}^3 \text{ sec}^{-1}$

`effective_volume` - if None, the standard thermalization model of K. Griest and D. Seckel is used. For alternative scenarios, see WimPyC paper(arXiv:2510.21185).

Output:

WIMP annihilation rate in sec^{-1}
